



COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

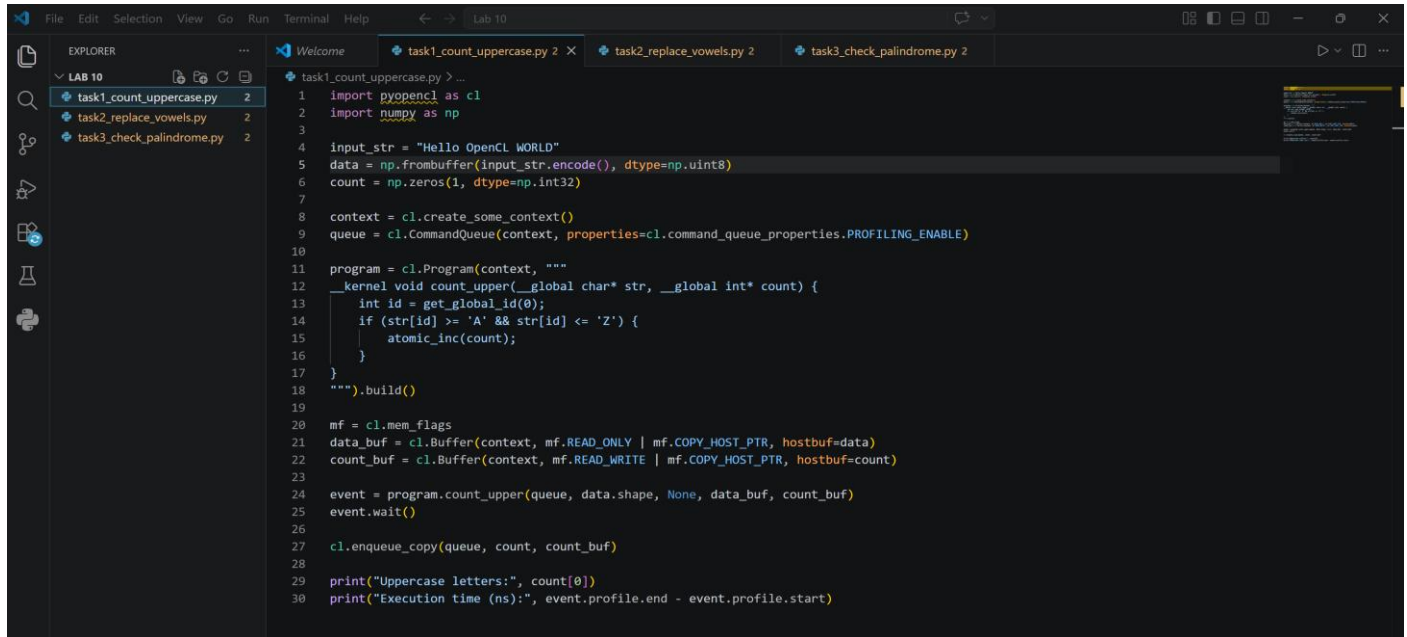
SUBMITTED BY

NAME : RIFAQ AJMAL
REG NO : 23MDBCS 435
SECTION : CS-A
SEMESTER : 6th

Date: 22/4/2026

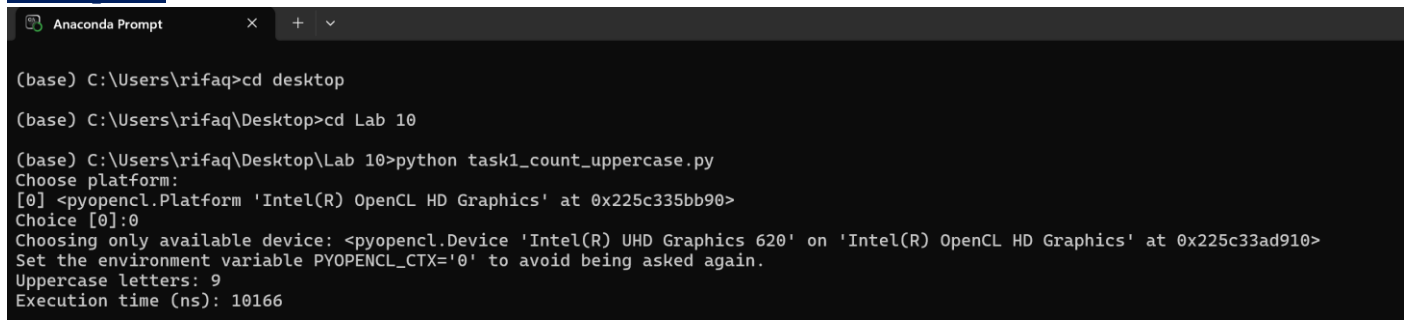
Lab 10

Task 1



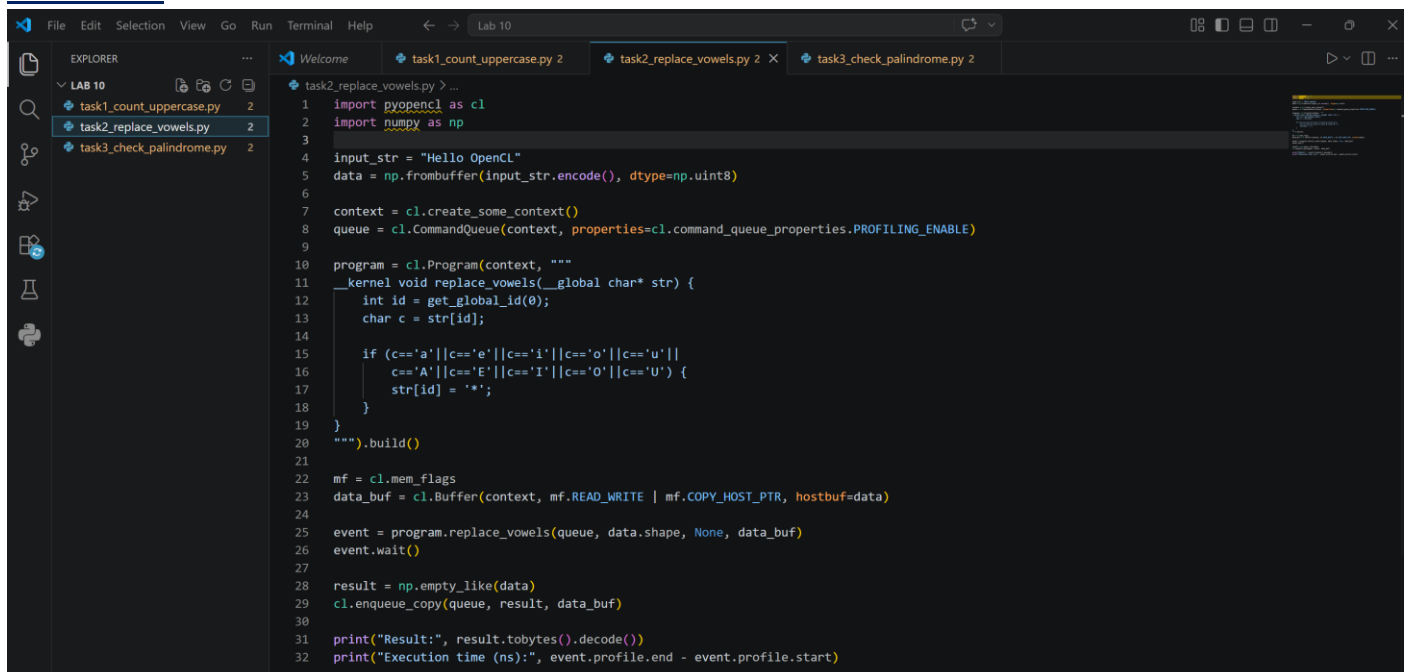
```
1 import pyopencl as cl
2 import numpy as np
3
4 input_str = "Hello OpenCL WORLD"
5 data = np.frombuffer(input_str.encode(), dtype=np.uint8)
6 count = np.zeros(1, dtype=np.int32)
7
8 context = cl.create_some_context()
9 queue = cl.CommandQueue(context, properties=cl.command_queue_properties.PROFILING_ENABLE)
10
11 program = cl.Program(context, """
12     kernel void count_upper(__global char* str, __global int* count) {
13         int id = get_global_id(0);
14         if (str[id] >= 'A' && str[id] <= 'Z') {
15             atomic_inc(count);
16         }
17     }
18 """).build()
19
20 mf = cl.mem_flags
21 data_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=data)
22 count_buf = cl.Buffer(context, mf.READ_WRITE | mf.COPY_HOST_PTR, hostbuf=count)
23
24 event = program.count_upper(queue, data.shape, None, data_buf, count_buf)
25 event.wait()
26
27 cl.enqueue_copy(queue, count, count_buf)
28
29 print("Uppercase letters:", count[0])
30 print("Execution time (ns):", event.profile.end - event.profile.start)
```

Output



```
(base) C:\Users\rifaq>cd desktop
(base) C:\Users\rifaq\Desktop>cd Lab 10
(base) C:\Users\rifaq\Desktop\Lab 10>python task1_count_uppercase.py
Choose platform:
[0] <pyopencl.Platform 'Intel(R) OpenCL HD Graphics' at 0x225c335bb90>
Choice [0]:0
Choosing only available device: <pyopencl.Device 'Intel(R) UHD Graphics 620' on 'Intel(R) OpenCL HD Graphics' at 0x225c33ad910>
Set the environment variable PYOPENCL_CTX='0' to avoid being asked again.
Uppercase letters: 9
Execution time (ns): 10166
```

Task 2



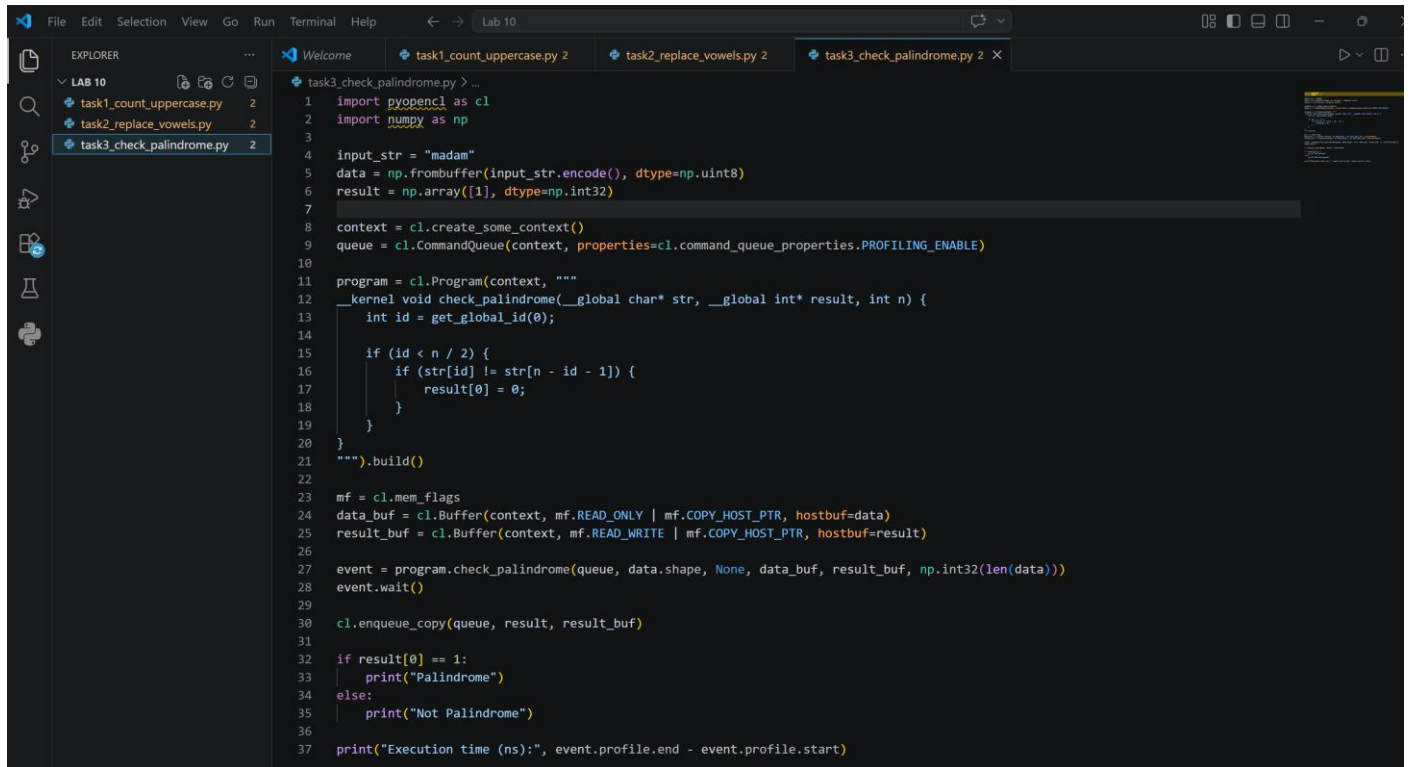
```
1 import pyopencl as cl
2 import numpy as np
3
4 input_str = "Hello OpenCL"
5 data = np.frombuffer(input_str.encode(), dtype=np.uint8)
6
7 context = cl.create_some_context()
8 queue = cl.CommandQueue(context, properties=cl.command_queue_properties.PROFILING_ENABLE)
9
10 program = cl.Program(context, """
11     kernel void replace_vowels(__global char* str) {
12         int id = get_global_id(0);
13         char c = str[id];
14
15         if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
16             c=='A' || c=='E' || c=='I' || c=='O' || c=='U') {
17             str[id] = '*';
18         }
19     }
20 """).build()
21
22 mf = cl.mem_flags
23 data_buf = cl.Buffer(context, mf.READ_WRITE | mf.COPY_HOST_PTR, hostbuf=data)
24
25 event = program.replace_vowels(queue, data.shape, None, data_buf)
26 event.wait()
27
28 result = np.empty_like(data)
29 cl.enqueue_copy(queue, result, data_buf)
30
31 print("Result:", result.tobytes().decode())
32 print("Execution time (ns):", event.profile.end - event.profile.start)
```

Output

```
(base) C:\Users\rifaa\Desktop\Lab 10>python task2_replace_vowels.py
Choose platform:
[0] <pyopencl.Platform 'Intel(R) OpenCL HD Graphics' at 0x1aa00115390>
Choice [0]:0
Choosing only available device: <pyopencl.Device 'Intel(R) UHD Graphics 620' on 'Intel(R) OpenCL HD Graphics' at 0x1aa00148b10>
Set the environment variable PYOPENCL_CTX='0' to avoid being asked again.
Result: H*ll* *p*nCL
Execution time (ns): 13666

(base) C:\Users\rifaa\Desktop\Lab 10>
```

Task 3



```
task3_check_palindrome.py > ...
1  import pyopencl as cl
2  import numpy as np
3
4  input_str = "madam"
5  data = np.frombuffer(input_str.encode(), dtype=np.uint8)
6  result = np.array([1], dtype=np.int32)
7
8  context = cl.create_some_context()
9  queue = cl.CommandQueue(context, properties=cl.command_queue_properties.PROFILING_ENABLE)
10
11  program = cl.Program(context, """
12  _kernel void check_palindrome(__global char* str, __global int* result, int n) {
13      int id = get_global_id(0);
14
15      if (id < n / 2) {
16          if (str[id] != str[n - id - 1]) {
17              result[0] = 0;
18          }
19      }
20  }
21  """).build()
22
23  mf = cl.mem_flags
24  data_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=data)
25  result_buf = cl.Buffer(context, mf.READ_WRITE | mf.COPY_HOST_PTR, hostbuf=result)
26
27  event = program.check_palindrome(queue, data.shape, None, data_buf, result_buf, np.int32(len(data)))
28  event.wait()
29
30  cl.enqueue_copy(queue, result, result_buf)
31
32  if result[0] == 1:
33      print("Palindrome")
34  else:
35      print("Not Palindrome")
36
37  print("Execution time (ns):", event.profile.end - event.profile.start)
```

Output

```
(base) C:\Users\rifaa\Desktop\Lab 10>python task3_check_palindrome.py
Choose platform:
[0] <pyopencl.Platform 'Intel(R) OpenCL HD Graphics' at 0x25fc7aaee90>
Choice [0]:0
Choosing only available device: <pyopencl.Device 'Intel(R) UHD Graphics 620' on 'Intel(R) OpenCL HD Graphics' at 0x25fc7ad2890>
Set the environment variable PYOPENCL_CTX='0' to avoid being asked again.
Palindrome
Execution time (ns): 9499

(base) C:\Users\rifaa\Desktop\Lab 10>
```